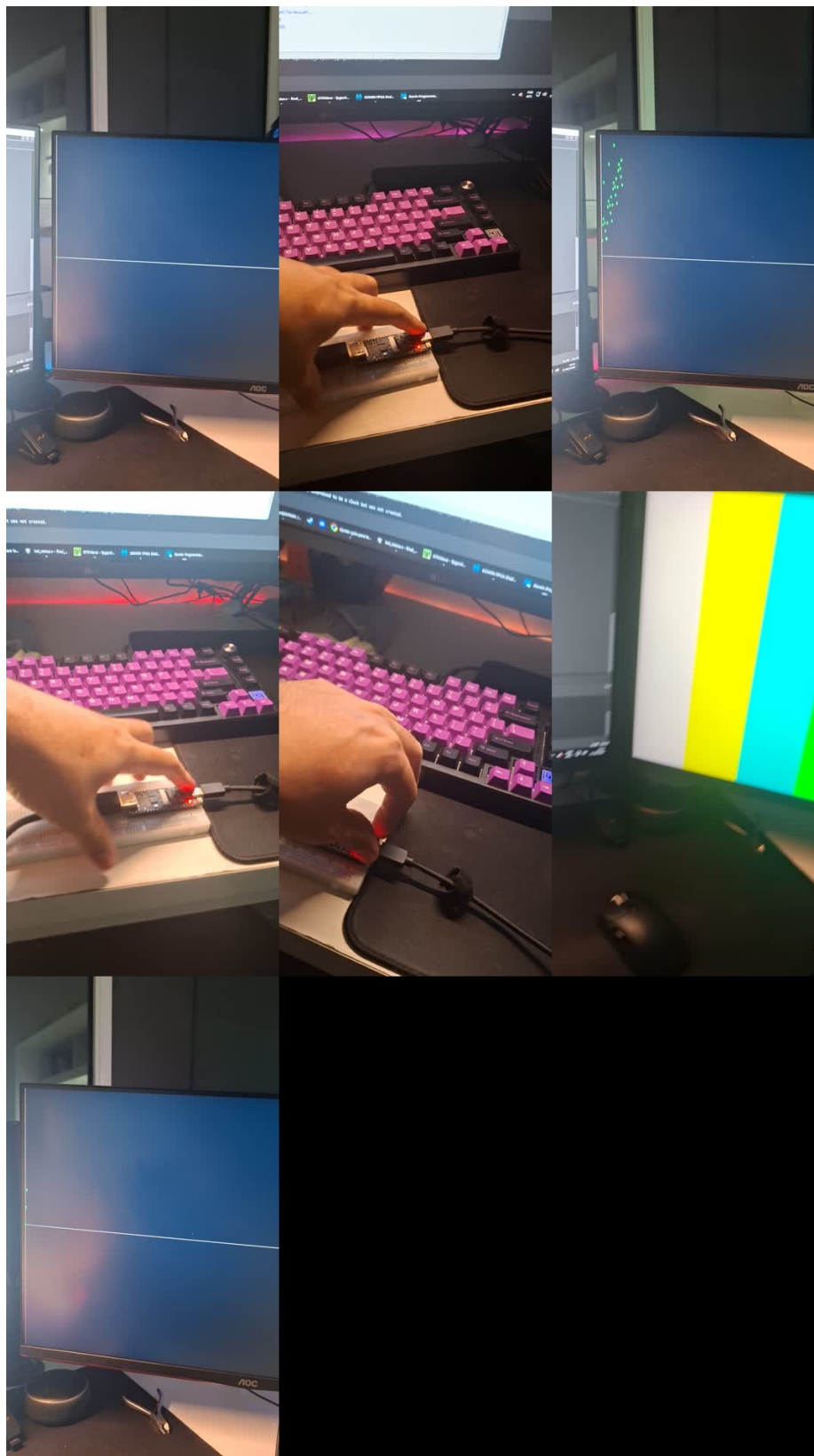


Apostila de Estudo do Projeto ARM64 + FPGA

Do zero absoluto ao projeto final: Tang Nano 4K, QEMU, Verilog, Assembly AArch64 e HDMI
Material reformulado para alguém que nunca estudou eletrônica digital, FPGA, Assembly ou protocolos.

Base documental: relatórios técnicos TP1, TP2, TP3, TP4 e TP5; guia do projeto final; evidências e comportamento final do protótipo.

Regra de ouro desta versão: Nenhuma sigla importante aparece como se você já devesse conhecê-la. Primeiro vem a ideia simples; depois a analogia; depois o nome técnico; só então o código e o papel no projeto.



Protótipo final real: Tang Nano 4K controlando a saída HDMI e exibindo modos/telemetria.

Didática adotada: linguagem simples, progressão em pequenos passos, analogias do cotidiano, repetição consciente, exemplos concretos do próprio projeto e perguntas de banca com resposta em duas camadas: simples e técnica.

Como estudar esta apostila sem se perder

O maior problema da versão anterior não era a quantidade de conteúdo. Era a ordem. Ela dizia que começava do zero, mas apresentava termos como UART, CRC, MAC, BSRAM, PLL e TMD5 muito cedo. Para quem nunca viu esses nomes, isso transforma o projeto em uma sopa de siglas. Nesta versão, a ordem foi invertida: primeiro você entende o problema humano; depois aprende as peças digitais; por fim recebe os nomes técnicos.

Uma promessa: Se você encontrar uma palavra técnica e ainda não souber explicar em linguagem comum o que ela faz, não avance. Volte ao quadro "Analogia" daquele conceito.

O método de três camadas

Camada	Pergunta	Exemplo com MAC
1 - Criança de 10 anos	O que isso faz, sem siglas?	É uma calculadora que multiplica dois números e soma o resultado a uma conta que já estava guardada.
2 - Aluno de engenharia	Qual é o nome e qual a regra?	MAC significa Multiply-Accumulate. O produto entra em um acumulador e pode gerar overflow.
3 - Projeto	Onde está no meu código e o que prova?	mac_unit.v; tb_mac_unit; resultado armazenado na telemetry_bram; overflow altera o LED.

Mapa de leitura

1. Comece pela história do projeto e pela demonstração final. Você precisa saber o que construiu antes de estudar como.
2. Aprenda o alfabeto digital: bit, byte, clock, reset, sinais, memória e estados.
3. Entenda a diferença entre CPU ARM64 e FPGA.
4. Aprenda Verilog, simulação e Gowin.
5. Só depois entre em vídeo, MAC, memória e comunicação.
6. Estude a evolução TP1 -> TP5 para entender por que a arquitetura mudou.
7. Finalize com o projeto final, os testes, as limitações e as perguntas da banca.

Parte 0 - Antes de qualquer sigla: o que este projeto realmente faz?

1. A história do projeto em linguagem comum

Imagine uma pequena máquina de laboratório. Ela recebe pedidos, verifica se o pedido faz sentido, executa uma conta, guarda alguns resultados e mostra informações em uma tela. Essa é a essência do projeto final.

O projeto possui dois mundos. No primeiro mundo existe um programa para um processador ARM64. Como o Raspberry Pi físico não estava disponível nas etapas finais, esse programa foi executado dentro do QEMU, que imita uma máquina ARM64. No segundo mundo existe a Tang Nano 4K, uma placa com FPGA. A FPGA é programada para virar um conjunto de circuitos digitais que trabalham ao mesmo tempo.

ARM64 no QEMU cria pedidos	Protocolo organiza e confere	FPGA executa e guarda	HDMI + LED mostram o estado
-------------------------------	---------------------------------	--------------------------	--------------------------------

O que a tela significa: As barras, o xadrez, o gradiente e o gráfico não são o objetivo isolado. Eles são um painel visual para provar que o estado interno da FPGA mudou e que a cadeia de controle, processamento e vídeo está viva.

1.1 O resultado final que você já viu

Modo	O que aparece no HDMI	O que isso prova
00	Barras de cores	O gerador de pixels está produzindo o modo 0 e o HDMI está sincronizado.
01	Xadrez	O active_mode mudou para 1 e o gerador de padrão respondeu.
10	Gradiente RGB	O modo 2 está ativo e as coordenadas de pixel estão sendo usadas para calcular cor.
11	Tela de telemetria com eixo/gráfico	O modo de observação dos resultados do MAC está ativo e lê a memória de telemetria.

Os dois pequenos quadrados no canto superior direito são os dois bits de active_mode. Eles tornam visível o número binário do modo. O LED onboard ficou reservado para eventos: uma piscada curta para comando aceito, duas piscadas para erro de comando/protocolo e piscada rápida contínua para erro grave de overflow até um CLEAR.

Pergunta que o professor pode fazer: Então o projeto é só apertar botão e trocar tela?

Não. O botão é apenas uma forma física simples de gerar comandos na placa. Por trás dele existem sincronização, debounce, criação de comando, executor, registradores de estado, MAC, memória e um pipeline HDMI. A troca de tela é a evidência visível de que essa cadeia funcionou.

1.2 O que é físico e o que é simulado

Parte	Onde roda	É física?	O que comprova
Programa Assembly AArch64	QEMU no computador	Não	Que o binário ARM64 executa e monta os pacotes esperados.
Protocolo ARM64 -> FPGA	Co-simulação com testbench	Não é um fio real	UART, parser, CRC, sequência, executor e respostas funcionam logicamente juntos.
FPGA final	Tang Nano 4K	Sim	Clocks, botões, executor, MAC, memória, LED e HDMI funcionam no hardware.
Monitor HDMI	Monitor real	Sim	O fluxo de vídeo gerado pela FPGA chega corretamente ao monitor.

Limitação que você deve declarar: Não houve um Raspberry Pi físico enviando bytes por um cabo diretamente para a Tang Nano. A integração ARM64-FPGA foi comprovada por co-simulação; a FPGA foi comprovada fisicamente por botões, LED, MAC, memória e HDMI.

Parte 1 - O alfabeto digital: aprender a ler o que o hardware está dizendo

2. Bit: a menor resposta possível

Um bit é uma informação que só pode estar em um de dois estados. Nós escrevemos esses estados como 0 e 1. Em um circuito, esses números representam faixas de tensão que o hardware interpreta como nível baixo ou alto.

Analogia: Pense em um interruptor de luz: desligado ou ligado. O bit é a versão matemática dessa ideia.

Onde aparece no projeto: reset, valid, ready, overflow, cada bit de active_mode e cada bit que viaja pela UART são exemplos de bits.

Por que existe: O hardware digital precisa de símbolos simples para representar decisões, números e estados.

2.1 Byte e palavra

Oito bits juntos formam um byte. Um byte pode representar 256 combinações diferentes. Uma palavra é um grupo maior de bits usado naturalmente por uma arquitetura. No ARM64, os registradores gerais têm 64 bits.

Analogia: Um bit é uma letra; um byte é uma palavra curta feita de oito letras binárias; um registrador de 64 bits é como uma linha inteira do caderno.

Onde aparece no projeto: Cada campo do pacote final tem bytes; o payload ocupa até quatro bytes; registradores x0 a x30 do ARM64 têm 64 bits.

2.2 Binário, decimal e hexadecimal

O mesmo número pode ser escrito de várias formas. Decimal usa os dígitos 0 a 9. Binário usa apenas 0 e 1. Hexadecimal usa 0 a 9 e A a F. Hexadecimal é muito usado porque cada dígito hexadecimal representa exatamente quatro bits.

Analogia: É como escrever uma distância em metros ou centímetros: o valor é o mesmo, mas a forma de escrever muda.

Onde aparece no projeto: O início de pacote 0xA5, os comandos 0x01, 0x04 e os CRCs aparecem em hexadecimal porque dumps de bytes ficam mais fáceis de conferir.

0xA5 = 1010 0101 em binário = 165 em decimal

2.3 Sinal, fio lógico, entrada e saída

Um sinal é um valor digital que pode mudar ao longo do tempo. Em Verilog, sinais ligam módulos. Uma entrada recebe informação; uma saída entrega informação.

Analogia: Imagine tubos entre caixas. A caixa não precisa saber como o tubo foi construído; só precisa saber o que chega e o que sai.

Onde aparece no projeto: clk, rst_n, cmd_valid, active_mode, led_pin e tmds_d_p são sinais do projeto.

2.4 Clock: o tic-tac do circuito

Clock é um sinal que alterna repetidamente entre 0 e 1. Muitos elementos da FPGA usam uma borda do clock como o momento oficial para atualizar valores guardados.

Analogia: É o apito do professor dizendo: "agora todos trocam de página". Cada aluno pode estar fazendo uma tarefa diferente, mas o instante da mudança é comum.

Onde aparece no projeto: A Tang Nano fornece 27 MHz. O projeto deriva um clock rápido de aproximadamente 126 MHz e um clock de pixels de aproximadamente 25,2 MHz.

Por que existe: Sem uma referência de tempo comum, contadores, vídeo, UART e máquinas de estado não saberiam quando avançar.

Se der errado: Clocks incorretos podem causar vídeo sem sincronismo, UART interpretando bits no momento errado e falhas de timing.

2.5 Frequência e período

Frequência diz quantas vezes algo acontece por segundo. 27 MHz significa cerca de 27 milhões de ciclos por segundo. Período é o tempo de um ciclo. Quanto maior a frequência, menor o período.

Analogia: Uma torneira que pinga 10 vezes por segundo tem frequência de 10 Hz. O tempo entre duas gotas é o período.

Onde aparece no projeto: O arquivo final.sdc informa ao Gowin que o clock de entrada de 27 MHz tem período de aproximadamente 37,037 ns.

2.6 Reset: voltar para um começo conhecido

Reset força registradores, contadores e estados internos para valores definidos. Isso evita que o sistema comece em uma situação aleatória.

Analogia: É o botão "novo jogo". Você não precisa adivinhar em qual fase ficou; volta ao início.

Onde aparece no projeto: reset_sync libera o restante do projeto apenas depois que o PLL está estável; CLEAR também zera partes do estado funcional como MAC e telemetria.

2.7 Estado e registrador

Estado é uma informação que precisa ser lembrada de um instante para o outro. Um registrador é um conjunto de elementos que guarda bits entre clocks.

Analogia: Um placar de jogo é estado: o ponto continua valendo mesmo quando ninguém está marcando um novo ponto.

Onde aparece no projeto: active_mode, pending_mode, acumulador do MAC, contadores de telemetria e estados da UART são informações guardadas.

Pergunta que o professor pode fazer: Qual é a diferença entre um valor combinacional e um valor guardado?

Um valor combinacional depende apenas das entradas atuais. Um valor guardado continua existindo de um clock para o outro porque está em registradores ou memória.

Parte 2 - CPU, ARM64, FPGA e QEMU: quem é quem?

3. Sistema embarcado

É um sistema computacional construído para uma função específica dentro de um equipamento. Em vez de ser um computador genérico para qualquer programa, ele existe para cumprir tarefas definidas.

Analogia: Uma máquina de lavar tem um computador pequeno que lê botões, controla motor, válvulas e tempo. Ele não precisa editar planilhas.

Onde aparece no projeto: Nos TPs, o sistema embarcado foi sendo construído para receber comandos, controlar lógica digital, processar números e produzir vídeo.

3.1 CPU

CPU é um processador que executa instruções armazenadas em memória. Ela busca uma instrução, entende o que deve fazer, executa e avança.

Analogia: É como uma pessoa seguindo uma receita passo a passo.

Onde aparece no projeto: O lado ARM64 é o lado de software: monta dados, percorre tabelas, converte números, cria pacotes e executa rotinas.

3.2 ARM64 e AArch64

ARM64 é uma arquitetura de processador de 64 bits. AArch64 é o estado de execução e o conjunto de instruções de 64 bits usado nos programas Assembly do projeto.

Analogia: É como dizer que um veículo pertence a uma família e usa um determinado conjunto de comandos no painel.

Onde aparece no projeto: Os arquivos .s dos TPs usam instruções como MOV, LDR, STR, ADD, MUL, CMP, B, BL, SVC, STP e LDP.

3.3 Assembly

Assembly é uma forma textual muito próxima das instruções que a CPU entende. Cada linha geralmente representa uma operação simples: mover um valor, somar, comparar, saltar ou acessar memória.

Analogia: Em vez de dizer "faça um sanduíche", você escreve "pegue o pão", "pegue o queijo", "coloque o queijo", "feche o pão".

Onde aparece no projeto: TP1 calculou endereços de pixel; TP2 preencheu buffer; TP3 praticou loops, condições, tabela de salto e parser; TP4 adicionou aritmética avançada e NEON; TP5 organizou tudo em módulos e biblioteca.

3.4 FPGA

FPGA é um chip reconfigurável. Você não entrega a ele uma sequência de instruções para executar como uma CPU. Você descreve circuitos e a ferramenta configura recursos internos para que esses circuitos existam fisicamente.

Analogia: Uma CPU é um cozinheiro seguindo uma receita. Uma FPGA é uma cozinha montada com várias estações trabalhando ao mesmo tempo: uma corta, outra frita, outra lava pratos.

Onde aparece no projeto: UART, parser, MAC, memória, controle e vídeo podem funcionar em paralelo na Tang Nano 4K.

Por que existe: Vídeo e interfaces digitais precisam de comportamento previsível ciclo a ciclo, algo muito natural para hardware dedicado.

3.5 QEMU

QEMU é um emulador. Ele permite executar um programa compilado para outra arquitetura, como AArch64, mesmo quando o computador principal não é um Raspberry Pi ARM64.

Analogia: É como um ator que segue as regras de outro personagem: não vira fisicamente o personagem, mas reproduz seu comportamento suficiente para testar a história.

Onde aparece no projeto: Do TP3 em diante, as rotinas ARM64 foram executadas e depuradas com qemu-aarch64 porque o Raspberry Pi físico não estava disponível.

Por que existe: QEMU permite validar software e instruções ARM64 sem depender do hardware físico.

Se der errado: QEMU não cria automaticamente fios GPIO, SPI ou UART reais até a Tang Nano.

3.6 CPU e FPGA juntas

A ideia do projeto é dividir responsabilidades. Software é bom para regras flexíveis, estruturas de dados e preparação de comandos. FPGA é boa para tarefas paralelas, temporizadas e contínuas.

Analogia: O escritório escreve o pedido; a fábrica executa máquinas em paralelo.

Onde aparece no projeto: No projeto final, ARM64 monta pacotes; a co-simulação prova o protocolo; na placa, o executor, MAC, memória e HDMI rodam na FPGA.

Pergunta que o professor pode fazer: Por que não fazer tudo no ARM64?

Seria possível fazer muitas tarefas em software, mas o objetivo da disciplina é exercitar hardware digital. A FPGA permite demonstrar temporização, paralelismo, memória interna, DSP e geração contínua de vídeo.

Parte 3 - Verilog: desenhando circuitos com texto

4. Verilog não é um programa comum

Verilog é uma linguagem de descrição de hardware. Quando duas partes independentes do código descrevem dois blocos de circuito, os dois podem funcionar ao mesmo tempo.

Analogia: Um desenho elétrico mostra vários fios e componentes ativos ao mesmo tempo. A ordem em que você leu o papel não obriga a eletricidade a esperar.

Onde aparece no projeto: Todos os módulos .v da pasta fpga/rtl e da cópia Gowin descrevem circuitos que serão simulados ou sintetizados.

4.1 Módulo

Um módulo é uma peça de hardware com entradas, saídas e lógica interna. Módulos podem ser conectados dentro de outros módulos maiores.

Analogia: É uma peça de LEGO com encaixes bem definidos.

Onde aparece no projeto: sync2ff, debounce_pulse, button_commands, command_executor, mac_unit, telemetry_bram, led_status e pattern_stream são módulos.

4.2 Top-level

Top-level é o módulo que fica no topo da hierarquia. Ele conecta o circuito interno às portas físicas da placa.

Analogia: É a planta geral de uma casa, mostrando como os cômodos se conectam às portas e janelas externas.

Onde aparece no projeto: No projeto final, top_final é o top físico. O Gowin precisa saber disso para sintetizar a arquitetura correta.

4.3 Lógica combinacional

É uma lógica cuja saída depende das entradas atuais. Ela não precisa lembrar o que aconteceu antes.

Analogia: Uma calculadora simples de soma: muda os números, muda o resultado.

Onde aparece no projeto: O color_selector do TP1 e partes do pattern_generator/pattern_stream calculam cor a partir da posição do pixel.

4.4 Lógica sequencial

É uma lógica que guarda estado e normalmente muda esse estado em uma borda do clock.

Analogia: Um contador de voltas lembra quantas voltas já foram dadas.

Onde aparece no projeto: Contadores VGA, acumulador MAC, UART, debounce e registradores de comando são sequenciais.

```
always @(posedge clk) begin
    if (!rst_n) contador <= 0;
    else contador <= contador + 1;
end
```

4.5 wire, reg e largura [n:0]

wire representa uma conexão. reg, no Verilog tradicional, é uma variável que pode receber atribuição em um bloco always. A notação [7:0] significa oito bits, do bit 7 até o bit 0.

Analogia: wire é um fio entre duas caixas; reg é um espaço onde um bloco de lógica pode colocar um valor. A largura diz quantos fios existem em paralelo.

Onde aparece no projeto: cmd_id é 8 bits; cmd_payload é 32 bits; active_mode é 2 bits.

4.6 FSM - máquina de estados finitos

FSM é um circuito que só pode estar em um conjunto conhecido de estados e muda de um para outro seguindo regras.

Analogia: Um semáforo: verde -> amarelo -> vermelho -> verde. Ele não inventa uma cor nova no meio.

Onde aparece no projeto: TP3 construiu uma FSM de comandos com IDLE, captura, validação, execução, erro e espera de liberação. No projeto final, várias máquinas menores aparecem em UART, parser e lógica de status.

4.7 Handshake valid/ready

Handshake é uma pequena conversa entre dois blocos. Um bloco diz "tenho dado válido" e o outro diz "estou pronto". A transferência acontece quando as duas condições combinam.

Analogia: É como entregar uma caixa: você só solta quando a outra pessoa disser que está segurando.

Onde aparece no projeto: cmd_valid/cmd_ready e sinais de streaming de vídeo usam essa ideia para evitar perder dados.

Parte 4 - Entradas físicas: por que apertar um botão é mais difícil do que parece

5. Entrada assíncrona

Um botão físico não conhece o clock interno da FPGA. Ele pode mudar exatamente perto de uma borda do clock.

Analogia: Uma pessoa entra na sala no exato momento em que a foto da turma é tirada. Ela pode aparecer borrada ou no meio da porta.

Onde aparece no projeto: `USR_KEY_1` e `USR_KEY_2` entram em `sync2ff` antes de chegar à lógica principal.

5.1 Metaestabilidade

Quando um flip-flop recebe uma mudança de entrada perto demais da borda do clock, ele pode levar um pequeno tempo para resolver se o valor será 0 ou 1. Esse fenômeno é chamado metaestabilidade.

Analogia: É como uma moeda quase parando em pé antes de finalmente cair para cara ou coroa.

Onde aparece no projeto: `sync2ff` usa dois estágios para reduzir muito a chance de um valor incerto chegar ao restante do circuito.

5.2 Sincronizador de dois flip-flops

O primeiro flip-flop recebe o sinal externo; o segundo recebe a saída do primeiro no clock seguinte. Isso dá tempo para o primeiro se estabilizar.

Analogia: Duas catracas em sequência antes de entrar em uma área importante.

Onde aparece no projeto: `sync2ff.v` é usado para os dois botões do `top_final`.

5.3 Bounce e debounce

Um botão mecânico não muda de 0 para 1 uma única vez. Os contatos podem bater e oscilar rapidamente por alguns milissegundos. Debounce é o filtro que só aceita a mudança depois que ela permaneceu estável.

Analogia: Uma campainha com contato ruim poderia tocar várias vezes com um único toque. O filtro espera a campainha parar de tremer.

Onde aparece no projeto: `debounce_pulse` exige uma janela de aproximadamente 20 ms e gera `pressed_pulse` de um ciclo.

5.4 Comandos locais dos botões

Depois de sincronizar e filtrar, o projeto transforma pressões em comandos internos. Assim, o restante do sistema não precisa saber se o comando veio de um botão ou de um parser.

Analogia: Dois atendentes podem escrever o mesmo tipo de formulário. A fábrica recebe o formulário, não precisa conhecer quem o escreveu.

Onde aparece no projeto: `button_commands` gera `SET_MODE`, `MAC_STEP` e `CLEAR` e envia para o mesmo `command_executor` que recebe comandos na co-simulação.

Pergunta que o professor pode fazer: Por que isso é importante para a apresentação?

Porque demonstra que a arquitetura final tem um ponto comum de execução. A co-simulação testa pacotes ARM64; a placa usa botões; ambos chegam ao mesmo tipo de comando interno e exercitam a lógica central.

Parte 5 - Pixels, framebuffer e a evolução de VGA para HDMI

6. Pixel

Pixel é um ponto da imagem. Uma tela de 640 x 480 possui 640 posições horizontais em cada uma de 480 linhas visíveis.

Analogia: É um quadradinho minúsculo de um mosaico.

Onde aparece no projeto: TP1 começou escolhendo cores por coordenada; o projeto final continua calculando a cor do pixel atual.

6.1 Coordenadas X e Y

X indica a posição horizontal; Y indica a posição vertical. O canto superior esquerdo costuma ser X=0, Y=0.

Analogia: É como localizar uma cadeira por coluna e fila.

Onde aparece no projeto: color_selector, pattern_generator e pattern_stream usam coordenadas para decidir cor e posição do gráfico.

6.2 RGB

RGB representa as intensidades de vermelho, verde e azul. Misturando quantidades diferentes desses três canais, o monitor produz várias cores.

Analogia: Três lanternas coloridas apontando para o mesmo papel.

Onde aparece no projeto: TP1 usava canais de 4 bits no VGA; o pipeline HDMI final trabalha com canais de 8 bits antes da codificação TMDS.

6.3 Framebuffer

Framebuffer é uma região de memória que guarda a cor de cada pixel de uma imagem completa. Para mostrar a tela, o circuito lê essa memória na ordem correta.

Analogia: É como guardar uma fotografia inteira em um grande quadro quadriculado.

Onde aparece no projeto: TP1 e TP2 planejaram um framebuffer 640 x 480 de 1 byte por pixel, com o ARM calculando posições e a FPGA lendo a imagem.

Por que existe: A ideia é intuitiva para integrar software e vídeo.

Se der errado: Na Tang Nano 4K, a memória interna disponível é pequena demais para guardar 307.200 bytes de um quadro 8bpp. Por isso a arquitetura final mudou.

6.4 Geração procedural

Em vez de guardar todos os pixels, o circuito calcula a cor do pixel no momento em que ele precisa ser enviado.

Analogia: Em vez de guardar uma foto de um tabuleiro de xadrez, você guarda a regra: se a casa for par, use uma cor; se for ímpar, use outra.

Onde aparece no projeto: pattern_stream gera barras, xadrez e gradiente a partir de X, Y e active_mode. A BSRAM fica livre para telemetria.

6.5 Como calcular um endereço de pixel

Nos TPs iniciais, o ARM64 praticou a fórmula $addr = y * 640 + x$ para localizar um pixel em um framebuffer linear. Se você pensa em uma matriz 2D, cada linha possui 640 posições. Para chegar à linha y, pula-se y linhas completas; depois soma-se x para andar dentro da linha.

```
addr = y * 640 + x
Exemplo: x=10, y=2
addr = 2*640 + 10 = 1290
```

Por que esse aprendizado continua útil?: Mesmo que o framebuffer completo tenha sido abandonado no projeto final, essa conta ensina endereçamento linear de memória, uma habilidade usada em buffers, tabelas e telemetria.

7. VGA: o que os TPs 1 e 2 aprenderam

7.1 HSYNC e VSYNC

No VGA, o monitor precisa saber quando uma linha terminou e quando um quadro terminou. HSYNC marca a organização horizontal; VSYNC marca a organização vertical.

Analogia: É como um caderno: um sinal avisa "fim da linha" e outro avisa "fim da página".

Onde aparece no projeto: TP2 implementou contadores de 800 posições horizontais e 525 linhas totais para produzir 640 x 480 visível a cerca de 60 Hz.

7.2 Front porch, sync pulse e back porch

Além dos pixels visíveis existem intervalos de temporização que ajudam o monitor a se preparar entre regiões. Eles são chamados front porch, sync pulse e back porch.

Analogia: Depois de escrever uma linha, você deixa uma margem, faz um marcador e deixa outra margem antes da próxima linha.

Onde aparece no projeto: TP1/TP2 documentaram 640 visíveis + 16 front porch + 96 sync + 48 back porch = 800 períodos horizontais; verticalmente 480 + 10 + 2 + 33 = 525 linhas.

8. HDMI: a solução física final

8.1 HDMI digital

HDMI transporta vídeo digital. O projeto final não manda os sinais analógicos VGA diretamente. Ele produz pixels, codifica os dados e envia pares diferenciais de alta velocidade pelo conector HDMI da Tang Nano 4K.

Analogia: Em vez de falar com o monitor usando pulsos analógicos separados, você empacota a informação de vídeo em uma conversa digital muito rápida.

Onde aparece no projeto: O projeto reutiliza o pipeline oficial da Sipeed e injeta seu próprio pattern_stream.

8.2 TMDS

TMDS é a forma de codificação usada para transportar canais de vídeo no HDMI/DVI. Ela transforma cada byte de cor em uma palavra apropriada para transmissão serial, ajudando a controlar transições e equilíbrio elétrico.

Analogia: É como reescrever uma mensagem em um alfabeto próprio para viajar por um cabo rápido.

Onde aparece no projeto: svo_tmids e blocos oficiais fazem essa etapa. Você não precisou reescrever esse algoritmo.

8.3 Serialização e OSER10

Serializar significa pegar vários bits que existem em paralelo e enviá-los um após o outro muito rapidamente. OSER10 é um recurso que ajuda a enviar palavras de dez bits em série.

Analogia: Dez pessoas em uma fila passam por uma porta estreita uma de cada vez.

Onde aparece no projeto: O pipeline HDMI precisa de um clock rápido para serializar as palavras TMDS.

8.4 Par diferencial e ELVDS

Um par diferencial transporta um sinal usando dois fios complementares. O receptor observa a diferença entre eles, o que melhora robustez contra ruído.

Analogia: Duas pessoas carregam a mesma informação de maneiras opostas; o receptor compara as duas para ignorar parte do barulho do ambiente.

Onde aparece no projeto: A saída HDMI usa pares de dados e um par de clock através dos buffers diferenciais da FPGA.

8.5 PLL e CLKDIV

PLL é um bloco que gera clocks derivados de um clock de referência. CLKDIV divide um clock para obter uma frequência menor.

Analogia: Uma caixa de engrenagens pega uma rotação de entrada e produz rotações em velocidades diferentes.

Onde aparece no projeto: A placa fornece 27 MHz; o projeto usa o bloco oficial Gowin_PLLVR para gerar um clock rápido e Gowin_CLKDIV para obter aproximadamente 25,2 MHz de pixel.

Parte 6 - Números dentro da FPGA: Q8.8, MAC, overflow, DSP e memória

9. Inteiro com sinal e complemento de dois

Computadores precisam representar números negativos usando bits. O método mais comum é complemento de dois. O bit mais alto participa da representação do sinal.

Analogia: Imagine um contador circular: depois do maior positivo, o padrão continua representando os negativos.

Onde aparece no projeto: Amostras e coeficientes do MAC são valores signed de 16 bits.

9.1 Ponto fixo Q8.8

Q8.8 é uma maneira de representar números com parte fracionária sem usar ponto flutuante. Os 16 bits guardam um inteiro que deve ser interpretado como valor dividido por 256. Assim, 256 representa 1,0; 128 representa 0,5; -256 representa -1,0.

Analogia: É como guardar dinheiro em centavos. Você pode representar R\$ 12,34 como 1234 centavos e só lembrar onde fica a vírgula.

Onde aparece no projeto: sample_value, coeff_value, gain e resultados de telemetria usam esse tipo de escala.

Q8.8 -> valor real = inteiro_guardado / 256
256 -> 1,0
128 -> 0,5
384 -> 1,5
-192 -> -0,75

9.2 MAC - Multiply-Accumulate

MAC é uma operação que multiplica dois números e soma o resultado a um acumulador. Em vez de começar do zero em cada conta, ele mantém uma soma que cresce a cada passo.

Analogia: Você possui um cofrinho. Em cada rodada calcula quantidade x preço e joga esse valor dentro do cofrinho, somando ao que já estava lá.

Onde aparece no projeto: TP4 introduziu mac_unit com BRAM/DSP; TP5 revisou os casos de borda; o projeto final usa MAC_STEP e grava resultados de 16 bits na telemetria.

Por que existe: MAC é uma operação fundamental em filtros digitais, processamento de sinais e várias cargas numéricas.

9.3 Acumulador

É o registrador que guarda a soma parcial das operações MAC.

Analogia: É o visor de uma calculadora que continua somando resultados sem apagar o total anterior.

Onde aparece no projeto: accumulator_q1616 é o acumulador interno de 32 bits do MAC final.

9.4 Overflow

Overflow acontece quando um resultado matemático seria maior ou menor do que o intervalo que cabe no número de bits disponível.

Analogia: Um hodômetro com apenas três dígitos não consegue mostrar 1000 corretamente se não existir uma regra especial.

Onde aparece no projeto: mac_overflow sinaliza erro grave e faz o LED onboard piscar rapidamente até CLEAR.

9.5 Saturação

Saturação é uma forma de tratar overflow: em vez de deixar o número dar a volta, o circuito prende o resultado no maior ou menor valor permitido.

Analogia: Um termômetro que só vai até 50 graus continuaria mostrando 50 se o valor real passasse desse limite.

Onde aparece no projeto: TP4 e TP5 testaram saturação positiva e negativa na unidade MAC.

9.6 DSP

DSP, neste contexto, é um bloco aritmético dedicado dentro da FPGA, otimizado para multiplicações e operações relacionadas. Usá-lo evita construir toda a multiplicação com lógica genérica.

Analogia: Em vez de montar uma calculadora usando centenas de peças LEGO pequenas, a caixa já traz uma peça pronta de multiplicação.

Onde aparece no projeto: TP4 confirmou no relatório de síntese a inferência de MULTADDALU18X18. O projeto escolheu operandos 16 x 16 bits para combinar bem com esse recurso.

9.7 BRAM/BSRAM

É memória interna organizada em blocos dedicados da FPGA. Ela guarda mais dados com menos custo lógico do que tentar construir uma memória grande com flip-flops.

Analogia: É um armário pronto dentro da fábrica. Seria desperdício montar milhares de gavetas usando peças pequenas.

Onde aparece no projeto: TP4 usou memórias de amostras/coeficientes. O projeto final usa telemetry_bram para guardar até 1024 resultados Q8.8.

9.8 Telemetria

Telemetria é o registro de informações sobre o comportamento do sistema para que possamos observar o que aconteceu.

Analogia: É o painel de um carro mostrando velocidade, combustível e avisos.

Onde aparece no projeto: TP4/TP5 registraram status e payloads; no final, a BSRAM armazena resultados do MAC e o modo 11 os transforma em gráfico HDMI.

Parte 7 - Comunicação: como duas partes concordam sobre o significado dos bytes

10. Protocolo

Protocolo é um conjunto de regras para que quem envia e quem recebe interpretem os dados da mesma maneira.

Analogia: Uma carta possui remetente, destinatário, conteúdo e formato esperado. Se cada pessoa inventasse uma ordem diferente, seria difícil entender.

Onde aparece no projeto: O projeto final usa pacotes fixos de 10 bytes na co-simulação.

10.1 O pacote final de 10 bytes

Byte	Nome	Tradução simples
0	SOF = 0xA5	Marca de começo: "um pacote começa aqui".
1	Versão	Qual versão das regras está sendo usada.
2	Sequência	Número do pedido para detectar repetição.
3	Comando	Diz o que deve ser feito.
4	Length	Diz quantos bytes do payload têm significado.
5-8	Payload	Os dados do comando, até 4 bytes.
9	CRC	Valor de verificação contra corrupção.

10.2 Payload

Payload é a parte do pacote que carrega os dados úteis do comando.

Analogia: A caixa de encomenda tem etiqueta e lacre, mas payload é o produto dentro dela.

Onde aparece no projeto: SET_MODE leva o modo; SET_COLOR leva valores de cor; MAC_STEP leva dois operandos de 16 bits.

10.3 SPI

SPI é uma interface serial síncrona normalmente formada por clock, dados e seleção de dispositivo. Nos primeiros TPs ela foi planejada como ligação física ARM -> FPGA.

Analogia: É uma conversa em que um mestre bate o ritmo com um metrônomo próprio e os dados caminham junto com esse ritmo.

Onde aparece no projeto: TP1 e TP2 planejaram SPI para pixels; TP3 ainda tratava GPIO como preparação. A ligação física não foi concluída porque o Raspberry Pi não estava disponível.

10.4 UART

UART é uma forma de comunicação serial assíncrona. Os dois lados combinam antes a velocidade. Não existe um fio de clock acompanhando cada bit; o receptor usa seu próprio clock para amostrar no momento esperado.

Analogia: Duas pessoas combinam: "vou falar uma sílaba a cada segundo". Quem escuta usa seu próprio relógio.

Onde aparece no projeto: A co-simulação final usa uart_rx e uart_tx para transportar bytes a 115200 bits por segundo no testbench.

10.5 8N1

8N1 descreve um formato UART comum: 8 bits de dados, nenhuma paridade e 1 bit de parada. Um bit de início vem antes dos dados.

Analogia: Antes de falar uma palavra, você faz um sinal de "comecei"; fala oito pedaços; depois faz um sinal de "terminei".

Onde aparece no projeto: uart_rx detecta start bit, amostra 8 bits e confere stop bit. uart_tx faz o caminho inverso.

10.6 Baud rate / bits por segundo

A taxa 115200 indica a velocidade aproximada da transmissão serial. Cada bit dura um intervalo definido. O circuito calcula quantos clocks internos correspondem a esse intervalo.

Analogia: Se você fala 115200 pedacinhos de informação por segundo, cada pedacinho precisa ocupar uma fatia previsível de tempo.

Onde aparece no projeto: CLKS_PER_BIT relaciona o clock da FPGA com a taxa UART.

10.7 Checksum

Checksum é um valor calculado a partir dos dados para detectar mudanças. No TP4/TP5 foi usado um checksum aditivo simples: somar bytes e guardar parte do resultado.

Analogia: Some os números de uma lista e anote o total. Se a lista chegar diferente, existe boa chance de o total não bater.

Onde aparece no projeto: link.s dos TPs avançados usou checksum e log de telemetria.

10.8 CRC

CRC é uma verificação de erro mais estruturada, baseada em operações binárias e um polinômio. No projeto final foi usado CRC-8 com polinômio 0x07 sobre os bytes definidos do pacote.

Analogia: É um lacre matemático calculado a partir do conteúdo. Quem recebe recalcula o lacre e compara.

Onde aparece no projeto: packet.s gera CRC no ARM64; check_packets.py verifica; packet_parser recalcula na FPGA simulada.

10.9 Número de sequência

É um contador colocado no pacote para identificar a ordem e detectar se um pedido igual está sendo repetido indevidamente.

Analogia: Senhas de atendimento: pedido 41, depois 42. Se 42 aparecer duas vezes, você percebe que houve repetição.

Onde aparece no projeto: O parser guarda last_seq e rejeita sequência duplicada na co-simulação.

10.10 ACK e NACK

ACK é uma resposta de confirmação: "recebi e aceitei". NACK ou status de erro significa "não aceitei" e normalmente informa o motivo.

Analogia: É a confirmação de entrega de uma encomenda.

Onde aparece no projeto: response_builder e protocol_system geram respostas; o testbench confere ACK, CRC inválido e duplicata.

10.11 Parser

Parser é o bloco que recebe uma sequência de símbolos ou bytes e separa seus campos de acordo com as regras do protocolo.

Analogia: É uma pessoa abrindo um formulário e colocando cada informação na gaveta certa: nome, número, pedido, assinatura.

Onde aparece no projeto: packet_parser espera 0xA5, coleta bytes, calcula CRC, valida versão/comando/tamanho/sequência e só então levanta cmd_valid.

10.12 Executor de comandos

O executor recebe um comando já considerado válido e modifica o estado funcional do sistema.

Analogia: O fiscal confere o formulário; o executor é o setor que realmente realiza o pedido.

Onde aparece no projeto: command_executor trata SET_MODE, SET_COLOR, SET_GAIN, MAC_STEP, CLEAR, GET_STATUS e PING. Na placa, button_commands alimenta o mesmo executor.

Parte 8 - Assembly ARM64: os conceitos dos TPs explicados do começo

11. Registradores

Registradores são pequenos espaços de armazenamento dentro da CPU, muito rápidos, usados diretamente pelas instruções.

Analogia: São como as mãos do cozinheiro: cabe pouca coisa, mas estão sempre prontas para o próximo passo.

Onde aparece no projeto: Os TPs usam x0-x30 para 64 bits e w0-w30 para a parte baixa de 32 bits. x0/x1 também servem para argumentos e retornos de funções.

11.1 MOV, ADD, MUL, CMP e saltos

MOV coloca um valor em um registrador. ADD soma. MUL multiplica. CMP compara. Instruções de branch mudam o caminho de execução conforme o resultado.

Analogia: São verbos muito básicos: pegue, some, multiplique, compare, vá para outro passo.

Onde aparece no projeto: TP1 calculou $y * 640 + x$; TP2 usou loops duplos e decisões de cor; TP3 classificou valores e interpretou comandos.

11.2 LDR, STR, LDRB e STRB

LDR carrega dados da memória para registradores; STR grava registradores na memória. As versões com B trabalham com um byte.

Analogia: LDR é pegar um papel de uma gaveta; STR é colocar um papel na gaveta.

Onde aparece no projeto: TP2 preencheu tx_buffer com STRB e leu amostras com LDRB. TP3 simulou registradores GPIO através de LDR/STR.

11.3 Loop

Loop repete um conjunto de instruções enquanto uma condição não indicar o fim.

Analogia: Contar cadeiras de uma sala: verifica uma cadeira, avança, repete até chegar à última.

Onde aparece no projeto: TP1 percorreu 8 pixels; TP2 percorreu 640 x 480 posições; TP3 implementou sum_array; TP5 executou benchmarks com várias iterações.

11.4 Condicionais

Condicionais escolhem caminhos diferentes dependendo de comparações.

Analogia: Se estiver chovendo, leve guarda-chuva; senão, saia sem ele.

Onde aparece no projeto: TP2 escolheu borda, cruz e xadrez; TP3 implementou classify_value e parser.

11.5 Função, BL, retorno e pilha

Uma função é um bloco reutilizável. BL chama a função e guarda onde deve voltar. A pilha é uma área de memória usada para preservar dados temporários, endereços de retorno e registradores.

Analogia: Você marca a página do livro antes de consultar um apêndice; depois volta ao ponto marcado.

Onde aparece no projeto: TP3/TP4/TP5 usam convenção AAPCS64, STP/LDP para preservar registradores e x30 como link register.

11.6 ABI e caller-saved / callee-saved

ABI é um acordo sobre como funções trocam argumentos, retornos e registradores. Alguns registradores podem ser alterados livremente por uma função; outros precisam ser preservados.

Analogia: É uma regra de uso de uma sala compartilhada: algumas mesas podem ser reorganizadas, outras devem ficar exatamente como estavam.

Onde aparece no projeto: TP4 encontrou bugs reais quando registradores de retorno ou temporários não foram

preservados corretamente; TP5 corrigiu preservação no benchmark.

11.7 Syscall

Syscall é um pedido do programa ao sistema operacional para executar uma operação privilegiada, como escrever no terminal ou encerrar o processo.

Analogia: O programa pede ao porteiro do prédio para fazer algo que só o porteiro tem autorização para fazer.

Onde aparece no projeto: syscalls.s fornece rotinas de escrita e saída no ambiente Linux AArch64 emulado.

11.8 GDB

GDB é um depurador. Ele permite parar o programa, olhar registradores e memória e avançar instrução por instrução.

Analogia: É como apertar pause em um filme e examinar cada personagem e objeto da cena.

Onde aparece no projeto: TP2 a TP5 usaram GDB remoto conectado ao QEMU para comprovar valores, parser, GPIO simulado, conversões e pacotes.

11.9 Objdump e disassembly

Objdump consegue mostrar símbolos e instruções presentes nos arquivos compilados. Disassembly é a visualização das instruções de máquina de volta em forma legível.

Analogia: É abrir um brinquedo montado e conferir quais peças realmente foram usadas.

Onde aparece no projeto: TP3 usou make disasm; TP5 usou objdump para conferir símbolos exportados pela biblioteca.

11.10 Makefile

Makefile descreve como construir o projeto e quais arquivos dependem de quais outros. O comando make executa somente as etapas necessárias.

Analogia: É uma lista de receita com dependências: só refaz o bolo inteiro se os ingredientes relevantes mudaram.

Onde aparece no projeto: TP3 demonstrou build completo e incremental; TP5 separou alvos como lib, all, run, debug, disasm e lib_info; o projeto final usa make para ARM64 e simulações.

11.11 Biblioteca estática

Uma biblioteca estática é um arquivo que agrupa módulos compilados para que o linker possa incorporar as funções necessárias no programa final.

Analogia: É uma caixa de ferramentas pronta; o montador pega as ferramentas necessárias quando constrói o produto.

Onde aparece no projeto: TP5 gerou libtp5.a agrupando módulos de aritmética, IO, parsing, utilitários, link e NEON.

11.12 Macros

Macro é um trecho que o assembler pode expandir automaticamente, evitando repetir padrões de instruções.

Analogia: É um carimbo que reproduz um desenho inteiro com um único gesto.

Onde aparece no projeto: TP5 integrou macros parametrizadas ao projeto modular.

11.13 Parser de texto, itoa e atoi

Parser de texto identifica comandos em strings. itoa converte um número inteiro para caracteres; atoi faz o caminho oposto.

Analogia: itoa transforma o número 42 nas letras "4" e "2"; atoi lê essas letras e reconstrói o número 42.

Onde aparece no projeto: TP3 interpretou LED_ON/LED_OFF/STATUS; TP5 validou itoa e atoi_fn no GDB.

11.14 GPIO e mapeamento de memória

GPIO são pinos de entrada/saída de uso geral. Em muitos processadores, periféricos são controlados através de endereços de memória: escrever em certos endereços altera registradores de hardware.

Analogia: Imagine gavetas especiais: escrever um valor na gaveta "ligar" acende uma saída; ler a gaveta "nível" informa o estado de um pino.

Onde aparece no projeto: TP3 estudou GPFSEL, GPSET, GPCLR e GPLEV do Raspberry Pi, mas simulou esses registradores

em um buffer porque o hardware físico não estava disponível.

11.15 Tabela de salto

Uma tabela de salto guarda endereços de rotinas. Um índice escolhe diretamente qual endereço será usado, evitando uma cadeia longa de comparações.

Analogia: Um índice de livro leva direto à página correta.

Onde aparece no projeto: TP3 implementou `jump_table_dispatch` com ADR/LDR/BR.

11.16 Lookup table

Uma tabela de consulta guarda respostas pré-calculadas. Em vez de refazer uma conta, o programa usa um índice e lê o resultado pronto.

Analogia: Uma tabuada impressa evita recalcular 7×8 toda vez.

Onde aparece no projeto: TP4 implementou `lookup_square` para quadrados de 0 a 15.

11.17 Aritmética multi-palavra

Quando um número é maior do que cabe em um registrador, ele pode ser dividido em partes. A soma precisa transportar o carry da parte baixa para a parte alta.

Analogia: Somar números grandes no papel: quando uma coluna passa de 9, você "vai um" para a próxima coluna.

Onde aparece no projeto: TP4 implementou `add128` usando ADDS e ADC para somar 128 bits com registradores de 64 bits.

11.18 Ponto flutuante

Ponto flutuante representa números com ampla faixa de magnitude usando sinal, expoente e fração. É diferente do ponto fixo Q8.8.

Analogia: É como notação científica: $1,23 \times 10^5$. A vírgula pode "flutuar" porque o expoente informa a escala.

Onde aparece no projeto: TP4 usou SCVTF e FCVTZS para converter entre inteiro e ponto flutuante.

11.19 Bitwise, máscaras, shifts e rotações

Operações bitwise trabalham diretamente com bits. Máscara seleciona partes de um valor; shift desloca bits; rotação move bits de uma ponta para a outra.

Analogia: Uma máscara de papel deixa visível apenas a parte do desenho que interessa.

Onde aparece no projeto: TP4 implementou `extract_field` e `rotate_left`; vários módulos de protocolo usam XOR e shifts.

11.20 SIMD e NEON

SIMD significa usar uma única instrução para operar vários elementos em paralelo. NEON é o conjunto vetorial do AArch64 usado nos TPs.

Analogia: Quatro caixas passam por uma esteira e uma máquina aplica a mesma operação às quatro de uma vez.

Onde aparece no projeto: TP4 somou quatro inteiros e multiplicou quatro floats em instruções vetoriais; TP5 manteve os módulos `neon_int` e `neon_float` na biblioteca. Isso foi aprendizado de otimização ARM, não o caminho principal do vídeo final.

Parte 9 - Ferramentas: o que cada programa faz na construção do projeto

Ferramenta	Tradução simples	Uso no projeto
GNU Assembler (as)	Traduz Assembly para objeto	Monta arquivos .s do ARM64.
GNU ld	Junta objetos em um executável	Gera packet_gen e outros binários.
QEMU	Imita a CPU ARM64	Executa AArch64 sem Raspberry Pi físico.
Python bridge	Converte e confere arquivos	bin_to_hex.py e check_packets.py.
Icarus Verilog (iverilog)	Compila Verilog para simulação	Gera arquivos .vvp dos testbenches.
vvp	Executa a simulação	Roda os testbenches e produz PASS/FAIL/VCD.
GTKWave	Visualiza sinais no tempo	Abre arquivos .vcd.
Gowin EDA	Transforma Verilog em hardware configurado	Síntese, Place & Route, timing, bitstream.
Gowin Programmer	Envia bitstream para a placa	Programa SRAM da Tang Nano 4K.
Make	Automatiza comandos e dependências	make clean, make all, make -C fpga/sim all.

12. Testbench

Testbench é um código usado somente para testar outro módulo em simulação. Ele cria entradas artificiais, observa saídas e pode imprimir PASS ou FAIL.

Analogia: É um simulador de treino para uma máquina antes de colocá-la no chão de fábrica.

Onde aparece no projeto: tb_mac_unit, tb_led_status, tb_pattern_stream e tb_arm_fpga_protocol são os testes finais.

12.1 VCD e waveform

VCD é um arquivo que registra como sinais digitais mudaram no tempo. GTKWave mostra esses sinais como formas de onda.

Analogia: É um eletrocardiograma da simulação: não mostra apenas o resultado final; mostra quando cada sinal mudou.

Onde aparece no projeto: Os relatórios TP2-TP5 usam waveforms para provar sincronismo, FSM, MAC, overflow e integração.

12.2 Síntese

Síntese transforma a descrição Verilog em uma rede de recursos que existem na FPGA: LUTs, flip-flops, memórias, DSPs e outros blocos.

Analogia: É transformar uma planta em uma lista real de materiais e conexões.

Onde aparece no projeto: No Gowin, Synthesize precisa terminar sem erros antes de Place & Route.

12.3 Place & Route

Place & Route escolhe onde cada recurso sintetizado será colocado dentro do chip e como as conexões serão roteadas.

Analogia: Depois de escolher todos os móveis de uma casa, alguém decide em qual cômodo cada um fica e por onde passam os fios.

Onde aparece no projeto: Essa etapa também permite analisar timing e gerar a implementação física.

12.4 Constraint CST

CST associa nomes lógicos do top-level aos pinos físicos da placa e configura características elétricas.

Analogia: É o mapa dizendo "a campainha lógica está ligada ao pino físico 10".

Onde aparece no projeto: final.cst conecta clk, botões, led_pin e pares HDMI aos pinos da Tang Nano.

12.5 Constraint SDC

SDC descreve requisitos de temporização, como a frequência do clock. Isso ajuda a ferramenta a verificar se os caminhos internos conseguem terminar no tempo esperado.

Analogia: É informar à fábrica qual é a velocidade da esteira para saber se cada estação consegue concluir a tarefa antes da próxima peça chegar.

Onde aparece no projeto: final.sdc declara o clock de 27 MHz. Um hdmi.sdc incompatível com onboard_led_bringup causou um erro porque tentava restringir objetos que não existiam naquele top.

12.6 Bitstream

Bitstream é o arquivo final que contém a configuração da FPGA. Ao programá-lo, o chip passa a se comportar como o circuito sintetizado.

Analogia: É o conjunto de instruções de montagem que reorganiza a fábrica antes de ligá-la.

Onde aparece no projeto: O Gowin gera um arquivo .fs e o Programmer grava em SRAM para testes.

Parte 10 - A história do projeto: o que cada TP acrescentou

13. TP1 - a ideia nasce: pixels, framebuffer e divisão ARM/FPGA

O TP1 desenhou a visão inicial de um Controlador VGA com Framebuffer ARM -> FPGA. O plano era o ARM gerar conteúdo, calcular endereços de pixels e enviar dados por SPI; a FPGA guardaria pixels em BRAM e cuidaria do sincronismo de vídeo. Como primeiro passo simples em Verilog, color_selector escolheu cores de barras com base na coordenada X. No Assembly, framebuffer_calc praticou $\text{addr} = y * 640 + x$.

O aprendizado central: Antes de integrar dispositivos, era preciso aprender duas ideias: como um pixel recebe uma cor e como uma posição 2D vira um endereço linear de memória.

Conceito do TP1	Sobreviveu no final?	Como mudou
Coordenadas de pixel e cores	Sim	pattern_stream continua calculando cores por X/Y.
ARM calcula dados	Sim em conceito	ARM64 final gera comandos/pacotes, não um quadro inteiro.
Framebuffer completo	Não	Memória interna não comporta 640x480x1 byte; final gera pixels proceduralmente.
SPI físico	Não concluído	Final usa UART na co-simulação e botões no hardware.
VGA físico	Substituído	Final usa HDMI da Tang Nano 4K.

14. TP2 - aprender tempo de vídeo e validar hardware

O TP2 desenvolveu vga_sync e pattern_generator. Os contadores horizontais e verticais ensinaram que uma imagem não é apenas uma matriz de cores: o monitor precisa receber os pixels no ritmo correto. O testbench mediu HSYNC, VSYNC e área visível. No ARM64, spi_emulator percorreu 307.200 posições e aplicou decisões de borda, cruz e xadrez. O Gowin introduziu top-level, pinagem, síntese, Place & Route e programação física.

O aprendizado central: Vídeo é uma tarefa de tempo real: não basta conhecer a cor; é preciso entregá-la no instante certo.

15. TP3 - transformar entradas em comportamento organizado

O TP3 colocou uma FSM no centro da experiência. A placa passou a reagir a botões e estados como IDLE, captura, validação, execução, erro e espera de liberação. Isso trouxe sincronização, debounce e tratamento de comando inválido. No lado ARM64, o TP praticou loops, condições, tabela de salto, parser de texto, GPIO mapeado em memória, GDB e Makefile incremental.

O aprendizado central: Um sistema digital real precisa controlar sequência, erros e entradas físicas imperfeitas; não é só fazer contas.

16. TP4 - a FPGA começa a processar números de verdade

O TP4 adicionou uma unidade MAC, memórias de bloco e uso de DSP. O objetivo deixou de ser apenas acender LED ou trocar estado: a FPGA passou a realizar processamento numérico com saturação e overflow. No ARM64 apareceram soma de 128 bits, conversão inteiro/float, lookup table, operações bitwise e NEON SIMD. A comunicação simulada recebeu checksum e telemetria.

O aprendizado central: A FPGA possui recursos especializados. Quando a arquitetura é escrita de forma adequada, o sintetizador consegue usar blocos de memória e multiplicadores dedicados em vez de construir tudo com lógica genérica.

17. TP5 - organização de software e integração parcial

O TP5 revisou o MAC com casos de borda, organizou o Assembly em vários módulos e criou uma biblioteca estática. O protocolo recebeu handshaking, checksum, marcador de fim de frame, benchmark e telemetria. A execução continuou no QEMU e a FPGA continuou física, mantendo a separação honesta entre o que foi emulado e o que foi medido na placa.

O aprendizado central: Projetos maiores precisam de organização, interfaces claras e testes de integração, não apenas

18. Projeto final - juntar os aprendizados e corrigir as limitações

O projeto final não copiou literalmente a arquitetura inicial. Ele escolheu as partes que faziam sentido com o hardware realmente disponível e corrigiu incompatibilidades. A saída tornou-se HDMI; o framebuffer completo foi substituído por geração procedural; o Raspberry Pi físico foi substituído por QEMU para o lado ARM64; a comunicação lógica final foi demonstrada por co-simulação UART; os botões permitem exercitar o executor na placa; o LED onboard substituiu os LEDs externos; e a BSRAM passou a guardar telemetria do MAC.

Plano antigo	Problema encontrado	Solução final
Framebuffer 640x480x8bpp na BSRAM	307.200 bytes excedem a memória interna disponível	Gerar pixels proceduralmente e reservar BSRAM para telemetria.
Raspberry Pi físico	Hardware não disponível	Executar AArch64 em QEMU.
SPI ARM->FPGA físico	Sem Raspberry Pi físico	Co-simulação de protocolo com UART + botões na placa.
VGA	Tang Nano 4K possui exemplo HDMI funcional e hardware apropriado	Reutilizar pipeline HDMI oficial da Sipeed.
3 LEDs externos	Bring-up físico externo consumiu tempo e não era essencial	HDMI mostra active_mode e LED onboard mostra status.

Parte 11 - Projeto final completo: acompanhar um comando do começo ao fim

19. O caminho lógico na co-simulação

main.s lista comandos	packet.s monta 10 bytes + CRC	commands.bin/hex ponte de arquivos	UART simulada bit a bit	packet_parser confere	command_executor executa
--------------------------	-------------------------------------	---------------------------------------	----------------------------	--------------------------	-----------------------------

O programa ARM64 começa em main.s. Ele chama a rotina de montagem de pacotes. Cada pacote recebe marcador 0xA5, versão, sequência, comando, tamanho, payload e CRC. O binário escreve 80 bytes: oito pacotes de dez bytes. bin_to_hex.py transforma cada byte em uma linha hexadecimal para o testbench. O testbench transmite esses bytes como uma UART real transmitiria: start bit, oito bits de dados e stop bit. uart_rx reconstrói bytes; packet_parser confere; command_executor executa; response_builder gera a resposta; uart_tx envia de volta.

O que o PASS final significa: "PASS integração ARM64 FPGA: 8 ACK, CRC e duplicata detectados" significa que o fluxo de software, bytes, UART, parser, validação e respostas funcionou junto na simulação. Não significa que houve um cabo serial real entre QEMU e a placa.

20. O caminho físico na Tang Nano 4K

Botões entrada física	sync2ff + debounce limpa entrada	button_commands cria comando	command_executor modifica estado	MAC/BSRAM processa e guarda	HDMI + LED mostram
--------------------------	-------------------------------------	---------------------------------	-------------------------------------	--------------------------------	-----------------------

Na placa, os botões substituem a origem do comando. Eles passam por sincronização e debounce. button_commands transforma as pressões nos mesmos tipos de comando internos. O executor não precisa saber se o pedido veio de um pacote ou de um botão. Isso é uma decisão importante de arquitetura: separar "como o comando chegou" de "o que o comando faz".

20.1 KEY1, KEY2 e KEY1+KEY2

Ação	Comando	Resultado esperado
Pressionar KEY1	SET_MODE	Avança entre 00, 01, 10 e 11. A mudança é aplicada em frame_start.
Pressionar KEY2	MAC_STEP	Executa uma multiplicação-acumulação usando a próxima amostra/coeficiente e grava telemetria.
Pressionar KEY1 e KEY2 juntos	CLEAR	Limpa acumulador/erro grave e reinicia parte do estado de telemetria.

20.2 Por que active_mode só muda no início do quadro?

O executor guarda primeiro o modo pedido em pending_mode. Quando frame_start sinaliza o começo de um novo quadro, active_mode recebe o valor pendente. Isso evita que metade de um quadro seja desenhada com um padrão e a outra metade com outro. É uma forma simples de fazer a atualização ocorrer em uma fronteira segura.

Analogia: Você não troca o cenário no meio da fotografia. Espera a próxima foto começar e então troca o cenário.

20.3 O que top_final conecta

Bloco	Função em linguagem simples
Gowin_PLLVR	Cria o clock rápido usado pelo pipeline HDMI.
Gowin_CLKDIV	Deriva o clock de pixels.
reset_sync	Só libera o sistema quando a base de clock está estável.
sync2ff	Protege a lógica contra entradas assíncronas dos botões.
debounce_pulse	Transforma um aperto físico em um evento limpo.
button_commands	Traduz eventos dos botões em comandos internos.
command_executor	Decide o que cada comando modifica.
mac_unit	Executa multiply-accumulate e detecta overflow.
telemetry_bram	Guarda os resultados recentes.
led_status	Transforma eventos em padrões de piscada.
pattern_stream	Escolhe a cor do pixel e desenha modo/telemetria.
svo_hdmi_final + SVO oficial	Leva o fluxo de pixels até TMDS/HDMI.

Parte 12 - O que cada tela e cada piscada está contando sobre o sistema

21. Modo 00 - barras de cores

É o modo inicial e mais simples de conferir. O circuito divide a largura da imagem em regiões e escolhe cores diferentes. Ele demonstra que coordenadas de pixel, geração RGB e pipeline HDMI estão funcionando. Os dois indicadores de modo ficam escuros porque 00 contém dois zeros.

22. Modo 01 - xadrez

O xadrez usa partes das coordenadas X e Y para alternar regiões. A ideia vem dos TPs iniciais: operações simples de bits conseguem gerar padrões geométricos sem guardar uma imagem pronta. O indicador binário mostra 01.

23. Modo 10 - gradiente

O gradiente transforma posição em intensidade de cor. Conforme X e Y avançam, os canais mudam. Isso torna muito visível que o circuito conhece a posição atual do pixel. O indicador mostra 10.

24. Modo 11 - telemetria

Esse é o modo que mais conecta processamento e vídeo. A memória possui resultados recentes do MAC. pattern_stream lê essas amostras e converte o valor numérico em altura do gráfico. A linha horizontal central funciona como referência. Quando ainda existem poucas amostras, a tela pode parecer quase vazia; isso não é erro. À medida que KEY2 executa MAC_STEP, pontos podem aparecer no gráfico. O indicador mostra 11.

25. LED onboard

Padrão	Significado	Prioridade
1 piscada curta (~150 ms)	Comando aceito	Baixa
2 piscadas médias	Erro de comando/protocolo	Média
Piscada rápida contínua	Erro grave / overflow do MAC	Alta, permanece até CLEAR

Por que um único LED é suficiente?: O HDMI já mostra o estado duradouro active_mode. O LED não precisa repetir essa informação; ele funciona como uma sirene de eventos e falhas.

Parte 13 - Validação: o que cada teste realmente prova

26. Unidade, integração, co-simulação e teste físico

Nível de teste	Exemplo	O que prova	O que NÃO prova sozinho
Unitário	tb_mac_unit	O MAC responde corretamente a casos isolados.	Não prova UART, botões ou HDMI.
Unitário	tb_led_status	Os padrões de piscada seguem as prioridades esperadas.	Não prova o LED físico nem pinagem.
Unitário/visual	tb_pattern_stream	active_mode escolhe padrões e indicadores.	Não prova TMDS no cabo real.
Integração	tb_arm_fpga_protocol	UART, parser, CRC, sequência, executor e respostas trabalham juntos.	Não prova enlace elétrico real QEMU->placa.
Físico	Gowin + Programmer + monitor	Clocks, pinagem, HDMI, botões, MAC, BSRAM e LED executam na Tang Nano.	Não prova que QEMU enviou bytes por um fio físico.

26.1 Os PASS que você obteve

PASS tb_mac_unit
PASS tb_led_status
PASS tb_pattern_stream: active_mode 00/01/10/11 visível no HDMI
PASS integracao ARM64 FPGA: 8 ACK, CRC e duplicata detectados

Essas quatro linhas são importantes porque cada uma cobre uma camada diferente. Juntas, elas mostram uma pirâmide: primeiro peças isoladas; depois comportamento visual; por fim integração de protocolo.

26.2 O que a síntese acrescenta à simulação

Simulação responde: "a lógica descrita se comporta como planejado neste modelo?". Síntese e Place & Route acrescentam perguntas físicas: "há recursos suficientes?", "o multiplicador virou DSP?", "a memória virou BSRAM?", "os caminhos fecham timing?", "as portas estão nos pinos corretos?".

26.3 O que a programação da placa acrescenta

Ao gravar o bitstream em SRAM e obter vídeo real, o projeto atravessa uma fronteira importante: clocks, pinos, sinalização elétrica e pipeline HDMI passaram a existir no chip real. Quando KEY1 muda os quatro modos e KEY2 alimenta telemetria, a validação física alcança a lógica central do protótipo.

Parte 14 - Erros reais encontrados e o que eles ensinam

Uma boa apresentação não precisa esconder que erros ocorreram. Em engenharia, depuração é parte do trabalho. O importante é explicar o que cada erro ensinou e como a hipótese foi testada.

Erro encontrado	Causa	Lição
check_packets.py: TypeError em raw % 10	Parênteses no lugar errado	Leia o tipo dos dados e confira a expressão que realmente foi executada.
0 PASS, 8 FALHA no CRC	return crc estava dentro do loop	Indentação em Python muda a lógica; o CRC estava correto nos pacotes, mas o verificador parava no primeiro byte.
iverilog: No such file or directory	Ferramenta não instalada no WSL	Diferencie erro de ambiente de erro de Verilog.
TA2003 em hdmi.sdc com onboard_led_bringup	Constraint HDMI aplicada a um top sem esses clocks/objetos	Constraints precisam corresponder à hierarquia ativa.
'clj' is not declared	Erro de digitação em button_commands.v	Primeiro erro do compilador costuma ser a raiz; erros seguintes podem ser cascata.
keyword 'ref' used in incorrect context	ref digitado no lugar de reg	SystemVerilog possui palavras reservadas; revise a declaração exata da porta.
LEDs externos não funcionaram	Caminho físico de protoboard não ficou confiável e multímetro falhou	Simplificar a arquitetura pode ser melhor do que insistir em uma evidência não essencial. HDMI assumiu active_mode; LED onboard ficou para status.

Mensagem de engenharia: Uma arquitetura boa também é aquela que você consegue provar com o hardware realmente disponível. Adaptar não significa esconder limitações; significa documentá-las e escolher uma validação honesta.

Parte 15 - Como explicar o projeto para o professor sem cair em uma sopa de siglas

27. Resposta de 30 segundos

"O projeto é um protótipo de sistema embarcado dividido entre software ARM64 e hardware FPGA. O ARM64, executado no QEMU, monta pacotes de comando e a integração do protocolo é validada por co-simulação. Na Tang Nano 4K, botões geram comandos equivalentes que passam pelo mesmo executor. A FPGA controla modos de vídeo, executa um MAC em ponto fixo, guarda telemetria em memória interna e mostra estado e resultados por HDMI, enquanto o LED onboard sinaliza comandos e erros."

27.1 Resposta de 2 minutos

"O trabalho começou nos TPs com uma ideia de controlador VGA e framebuffer. Ao longo das etapas foram introduzidos sincronismo de vídeo, máquinas de estado, tratamento de botões, Assembly ARM64, GPIO, processamento MAC, BRAM, DSP, checksum, telemetria e organização modular. Como o Raspberry Pi físico não estava disponível e um framebuffer completo não cabia na memória da Tang Nano 4K, a arquitetura final foi adaptada. O ARM64 passou a ser validado em QEMU e a integração do protocolo por co-simulação UART. A FPGA física usa botões como fonte local de comandos, executa o mesmo tipo de controle, faz MAC_STEP, guarda resultados em BSRAM e gera quatro modos HDMI. O modo 11 mostra a telemetria do MAC. Assim, a tela não é o objetivo isolado: ela é a interface visível de um sistema que recebe, valida, processa, armazena e apresenta dados."

28. Perguntas essenciais da banca

Pergunta que o professor pode fazer: O que é uma FPGA?

Resposta simples: é um chip que pode ser configurado para virar circuitos digitais específicos. Resposta técnica: o Verilog é sintetizado em recursos como LUTs, flip-flops, DSPs, BSRAM e blocos de IO que trabalham em paralelo.

Pergunta que o professor pode fazer: O que é MAC?

Resposta simples: uma calculadora que multiplica dois valores e soma o produto a um total acumulado. Resposta técnica: a unidade trabalha com operandos signed Q8.8, acumulador mais largo e sinalização de overflow/saturação.

Pergunta que o professor pode fazer: O que é UART?

Resposta simples: uma forma de enviar bits em série sem mandar um fio de clock junto; os dois lados combinam a velocidade. No projeto, ela foi usada na co-simulação do protocolo, não como enlace elétrico real QEMU->Tang Nano.

Pergunta que o professor pode fazer: Por que existe CRC?

Para detectar corrupção acidental de bytes. O emissor calcula um valor a partir dos dados; o receptor recalcula e compara. Se não bater, o pacote é rejeitado.

Pergunta que o professor pode fazer: Por que existe número de sequência?

Para identificar a ordem dos pedidos e evitar executar novamente uma retransmissão imediata ou pacote duplicado.

Pergunta que o professor pode fazer: O que é BRAM/BSRAM?

É memória interna dedicada da FPGA. No final ela guarda até 1024 resultados de telemetria do MAC, em vez de tentar guardar uma tela inteira.

Pergunta que o professor pode fazer: Por que não existe framebuffer completo?

Porque 640x480 com 1 byte por pixel exige 307.200 bytes, muito acima da BSRAM interna disponível na Tang Nano 4K. O final calcula pixels proceduralmente.

Pergunta que o professor pode fazer: Por que o projeto mudou de VGA para HDMI?

Os TPs ensinaram temporização de vídeo com VGA. No protótipo final foi mais adequado reutilizar o pipeline HDMI oficial da Tang Nano 4K, mantendo a mesma ideia de produzir pixels no tempo correto.

Pergunta que o professor pode fazer: O que o QEMU prova?

Prova execução do binário AArch64 e geração correta dos dados de software. Não prova características elétricas, latência real de GPIO/UART ou um cabo físico até a FPGA.

Pergunta que o professor pode fazer: O que a co-simulação prova?

Prova a integração lógica entre bytes gerados pelo ARM64 e os módulos Verilog de UART, parser, CRC, sequência, executor e resposta.

Pergunta que o professor pode fazer: Por que usar botões na placa?

Eles permitem exercitar fisicamente o executor e o datapath mesmo sem Raspberry Pi conectado. A origem do comando muda, mas a lógica executada na FPGA permanece a mesma.

Pergunta que o professor pode fazer: O que é DSP?

É um bloco aritmético dedicado da FPGA, eficiente para multiplicações. O TP4 confirmou que o sintetizador mapeou o multiplicador para um bloco MULTADDALU18X18.

Pergunta que o professor pode fazer: O que é overflow?

É quando o resultado matemático não cabe no número de bits disponível. No projeto, overflow do MAC é tratado como erro grave e sinalizado no LED.

Pergunta que o professor pode fazer: O que é debounce?

É um filtro temporal para impedir que um único aperto mecânico seja interpretado como vários apertos.

Pergunta que o professor pode fazer: O que é metaestabilidade?

É uma situação temporária de incerteza de um flip-flop quando uma entrada assíncrona muda perto da borda do clock. O sincronizador de dois flip-flops reduz a chance de o problema alcançar a lógica.

Pergunta que o professor pode fazer: Qual é o propósito das quatro telas?

Elas tornam `active_mode` observável e demonstram o pipeline de vídeo. O modo 11 vai além: transforma os resultados armazenados do MAC em telemetria visual.

Pergunta que o professor pode fazer: Então qual é a contribuição do projeto?

Integrar conceitos de software de baixo nível e hardware digital em um protótipo verificável: criação/validação de comandos, controle de estado, processamento numérico, memória, sinalização de erro e geração de vídeo.

Parte 16 - Glossário traduzido: do nome assustador para a ideia simples

Termo	Tradução simples
AArch64	Conjunto de instruções de 64 bits usado pelo ARM64.
ACK	Confirmação de que uma operação foi aceita.
ABI	Acordo de como funções usam registradores, pilha, argumentos e retornos.
Assembly	Linguagem textual próxima das instruções da CPU.
Bit	Informação com dois estados: 0 ou 1.
Bitstream	Arquivo que configura fisicamente a FPGA.
BRAM/BSRAM	Memória interna dedicada em blocos dentro da FPGA.
Byte	Grupo de 8 bits.
Checksum	Número simples calculado dos dados para detectar alterações.
CLKDIV	Bloco que divide um clock.
Clock	Sinal periódico que marca momentos de atualização do circuito.
Co-simulação	Teste no qual software e modelo de hardware participam do mesmo fluxo simulado.
Constraint	Regra física ou temporal entregue à ferramenta de implementação.
CRC	Verificação de erro baseada em operações polinomiais binárias.
CST	Arquivo de constraints físicas/pinagem do Gowin.
Debounce	Filtro que remove oscilações mecânicas de botão.
Determinismo	Previsibilidade do tempo e da ordem das respostas.
DSP	Bloco aritmético dedicado da FPGA.
ELVDS	Buffer usado para dirigir sinais diferenciais de alta velocidade.
FPGA	Chip reconfigurável em que Verilog vira circuitos digitais.
Frame	Um quadro completo de vídeo.
Framebuffer	Memória que guarda a cor de cada pixel de um quadro.
frame_start	Sinal que marca o início de um novo quadro.
FSM	Máquina de estados finitos.
GDB	Depurador para pausar e inspecionar CPU/memória.
GPIO	Pino de entrada/saída de uso geral.
GTKWave	Visualizador de waveforms VCD.
Handshake	Conversa de controle entre blocos para confirmar transferência.
HDMI	Interface digital de áudio/vídeo; aqui usada como saída de vídeo.
HSYNC	Sinal de sincronismo horizontal estudado nos TPs VGA.
Icarus Verilog	Ferramenta usada para compilar simulações Verilog.
Latency	Tempo entre uma entrada e a resposta.
Linker	Ferramenta que junta objetos e bibliotecas em um binário.
Lookup table	Tabela de respostas pré-calculadas acessadas por índice.
LUT	Pequena tabela lógica configurável da FPGA.
MAC	Multiply-Accumulate: multiplicar e acumular.
Makefile	Receita automatizada de build e testes.
Metaestabilidade	Incerteza temporária de flip-flop ao receber entrada fora do domínio de clock.
NEON	Extensão SIMD do AArch64.
Objdump	Ferramenta para inspecionar objetos, símbolos e disassembly.
OSER10	Serializador de dez bits usado no pipeline HDMI.

Overflow	Resultado maior/menor do que cabe na largura numérica.
Packet/Pacote	Grupo de bytes com campos definidos por protocolo.
Parser	Bloco que separa e valida campos de uma mensagem.
Payload	Dados úteis carregados dentro do pacote.
Pixel	Menor ponto endereçável da imagem.
Place & Route	Etapa que posiciona recursos e roteia conexões na FPGA.
PLL	Bloco que cria clocks derivados.
Q8.8	Ponto fixo de 16 bits com escala de 1/256.
QEMU	Emulador usado para executar AArch64 sem Raspberry Pi físico.
Register/Registrador	Armazenamento pequeno de bits associado ao clock ou CPU.
Reset	Ação de colocar o sistema em estado inicial conhecido.
RGB	Canais vermelho, verde e azul usados para formar cores.
SDC	Arquivo de constraints temporais.
Sequence number	Número que identifica ordem e repetição de pacotes.
SIMD	Uma instrução aplicada a vários elementos em paralelo.
SPI	Interface serial síncrona planejada nos TPs iniciais.
SVO	Conjunto de módulos de vídeo reutilizado do exemplo oficial da Sipeed.
Synthesis/Síntese	Transformação de Verilog em recursos físicos da FPGA.
Telemetry/Telemetria	Informações registradas para observar estado/resultados.
Testbench	Código de simulação usado para testar módulos.
TMDS	Codificação usada para transportar dados de vídeo HDMI/DVI.
Top-level	Módulo principal que conecta toda a hierarquia às portas externas.
UART	Comunicação serial assíncrona usada na co-simulação final.
VCD	Arquivo de mudanças de sinais usado pelo GTKWave.
Verilog	Linguagem de descrição de hardware.
VSYNC	Sinal de sincronismo vertical estudado nos TPs VGA.
Waveform	Gráfico dos sinais digitais ao longo do tempo.

Parte 17 - Plano de estudo em 7 dias

Dia	Meta	O que você deve conseguir explicar sem olhar
1	História do projeto + bit/byte/clock/reset	O que o projeto faz e por que a tela não é o objetivo inteiro.
2	CPU, FPGA, Verilog, FSM, debounce	Como um botão vira um comando confiável.
3	Pixels, framebuffer, VGA, HDMI	Por que o projeto mudou de framebuffer/VGA para geração procedural/HDMI.
4	Assembly, QEMU, registradores, memória, ABI, GDB	O que o lado ARM64 comprova e por que QEMU não é um cabo físico.
5	Q8.8, MAC, overflow, DSP, BSRAM	Como KEY2 pode gerar uma amostra de telemetria.
6	SPI, UART, pacote, checksum, CRC, sequência, ACK/NACK	Seguir um pacote de 10 bytes da criação até o executor.
7	TP1->TP5, projeto final e perguntas	Apresentar 5 minutos sem ler e responder limitações honestamente.

Técnica de estudo de 45 minutos

8. Leia um tópico e pare sempre que aparecer uma palavra que você não consegue traduzir.
9. Feche a apostila e explique o conceito em voz alta como se explicasse para alguém de 10 anos.
10. Abra o arquivo real do projeto relacionado ao conceito e localize o nome do sinal/módulo.
11. Responda duas perguntas da banca: primeiro em linguagem simples, depois acrescente o nome técnico.
12. No final, faça um resumo de uma frase: "isso existe porque..."

Parte 18 - Rastreabilidade: de onde veio cada bloco desta apostila

Esta apostila foi reorganizada a partir da sequência real dos trabalhos. Ela não trata todos os planos antigos como se fossem a arquitetura final. Quando um TP descreve algo que posteriormente foi substituído, o texto apresenta a ideia como etapa histórica e explica a solução final.

Fonte	Conceitos aproveitados
Relatório TP1	Visão ARM/FPGA, framebuffer, SPI planejado, timing VGA, color_selector, cálculo $\text{addr} = y * 640 + x$, primeiros conceitos AArch64.
Relatório TP2	vga_sync, pattern_generator, HSYNC/VSYNC, testbench, waveforms, constraints, fluxo Gowin, fill_buffer e decisões em Assembly.
Relatório TP3	FSM, sync2ff, debounce, Power-On Reset, botões, loops/condições/tabela de salto/parser, GPIO simulado, GDB, Makefile incremental.
Relatório TP4	MAC, saturação, BRAM, DSP, aritmética 128 bits, float, lookup, bitwise, NEON SIMD, checksum e telemetria.
Relatório TP5	MAC revisado, biblioteca estática, macros, itoa/atoi, protocolo com handshaking/checksum/EOF, benchmark, testes de integração e arquitetura consolidada.
Guia do projeto final v2	Co-simulação UART+CRC, pacote de 10 bytes, executor comum, HDMI, active_mode, telemetry_bram, LED onboard, top_final, constraints e validação final.

Versão final é a autoridade para a demonstração: Se um relatório antigo disser VGA, SPI físico, framebuffer completo ou LEDs externos, trate isso como etapa de evolução. A demonstração final usa HDMI, geração procedural, QEMU/co-simulação, botões e LED onboard.

Checklist final de compreensão

- Consigo explicar o projeto inteiro sem usar nenhuma sigla nos primeiros 30 segundos.
- Se eu disser FPGA, consigo definir FPGA imediatamente.
- Se eu disser MAC, consigo dar um exemplo numérico simples.
- Se eu disser UART, consigo desenhar start -> 8 bits -> stop e explicar que não existe fio de clock.
- Se eu disser CRC, consigo explicar que ele detecta corrupção e que o receptor recalcula.
- Consigo explicar por que framebuffer completo não cabe e por que o final gera pixels proceduralmente.
- Consigo distinguir o que foi simulado no QEMU/co-simulação do que foi testado fisicamente na Tang Nano.
- Consigo explicar o que cada uma das quatro telas mostra e o que o LED significa.
- Consigo seguir KEY1 e KEY2 até command_executor, MAC, BSRAM e HDMI.
- Consigo dizer o que os quatro PASS das simulações realmente provam.
- Consigo contar a evolução TP1 -> TP5 -> projeto final sem fingir que os planos antigos foram todos implementados fisicamente.

Documentos-base

- relatorio_tp1.pdf - TP1.
- relatorio_tecnico.pdf - TP2.
- relatório_tecnico.pdf - TP3.
- relatorio-tecnico(1).pdf - TP4.
- relatorio-tecnico(2).pdf - TP5.
- Guia_Didatico_Projeto_Final_ARM64_FPGA_v2_LED_Onboard.pdf - implementação final.
- apostila-at.pdf - versão anterior, usada como referência do que precisava ser reorganizado pedagogicamente.